

NdefRecord

Added in [API level 9](#) (/guide/topics/manifest/uses-sdk-element#ApiLevels)



[Kotlin](#) (/reference/kotlin/android/nfc/NdefRecord) | **Java**

```
public final class NdefRecord
extends Object (/reference/java/lang/Object) implements Parcelable
(/reference/android/os/Parcelable)
```

[java.lang.Object](#) (/reference/java/lang/Object)
↳ [android.nfc.NdefRecord](#)

Represents an immutable NDEF Record.

NDEF (NFC Data Exchange Format) is a light-weight binary format, used to encapsulate typed data. It is specified by the NFC Forum, for transmission and storage with NFC, however it is transport agnostic.

NDEF defines messages and records. An NDEF Record contains typed data, such as MIME-type media, a URI, or a custom application payload. An NDEF Message is a container for one or more NDEF Records.

This class represents logical (complete) NDEF Records, and can not be used to represent chunked (partial) NDEF Records. However [NdefMessage.NdefMessage\(byte\[\]\)](#) (/reference/android/nfc/NdefMessage#NdefMessage(byte[])) can be used to parse a message containing chunked records, and will return a message with unchunked (complete) records.

A logical NDEF Record always contains a 3-bit TNF (Type Name Field) that provides high level typing for the rest of the record. The remaining fields are variable length and not always present:

- *type*: detailed typing for the payload

- *id*: identifier meta-data, not commonly used
- *payload*: the actual payload

Helpers such as `NdefRecord.createUri`

([/reference/android/nfc/NdefRecord#createUri\(android.net.Uri\)](#)), `NdefRecord.createMime`

([/reference/android/nfc/NdefRecord#createMime\(java.lang.String,%20byte\[\]\)](#)) and

`NdefRecord.createExternal`

([/reference/android/nfc/NdefRecord#createExternal\(java.lang.String,%20java.lang.String,%20byte\[\]\)](#)) are

included to create well-formatted NDEF Records with correctly set tnf, type, id and payload fields, please use these helpers whenever possible.

Use the constructor `NdefRecord(short, byte[], byte[], byte[])`

([/reference/android/nfc/NdefRecord#NdefRecord\(short,%20byte\[\],%20byte\[\],%20byte\[\]\)](#)) if you know what you are doing and what to set the fields individually. Only basic validation is performed with this constructor, so it is possible to create records that do not confirm to the strict NFC Forum specifications.

The binary representation of an NDEF Record includes additional flags to indicate location with an NDEF message, provide support for chunking of NDEF records, and to pack optional fields. This class does not expose those details. To write an NDEF Record as binary you must first put it into an `NdefMessage` ([/reference/android/nfc/NdefMessage](#)), then call `NdefMessage.toByteArray()` ([/reference/android/nfc/NdefMessage#toByteArray\(\)](#)).

`NdefMessage` ([/reference/android/nfc/NdefMessage](#)) and `NdefRecord` ([/reference/android/nfc/NdefRecord](#)) implementations are always available, even on Android devices that do not have NFC hardware.

`NdefRecord` ([/reference/android/nfc/NdefRecord](#))s are intended to be immutable (and thread-safe), however they may contain mutable fields. So take care not to modify mutable fields passed into constructors, or modify mutable fields obtained by getter methods, unless such modification is explicitly marked as safe.

See also:

`NfcAdapter.ACTION_NDEF_DISCOVERED`

([/reference/android/nfc/NfcAdapter#ACTION_NDEF_DISCOVERED](#))

NdefMessage ([/reference/android/nfc/NdefMessage](#))

Summary

Constants

short	TNF_ABSOLUTE_URI (/reference/android/nfc/NdefRecord#TNF_ABSOLUTE_URI) Indicates the type field contains an absolute-URI BNF construct defined by RFC 3986.
short	TNF_EMPTY (/reference/android/nfc/NdefRecord#TNF_EMPTY) Indicates the record is empty.
short	TNF_EXTERNAL_TYPE (/reference/android/nfc/NdefRecord#TNF_EXTERNAL_TYPE) Indicates the type field contains an external type name.
short	TNF_MIME_MEDIA (/reference/android/nfc/NdefRecord#TNF_MIME_MEDIA) Indicates the type field contains a media-type BNF construct, defined by RFC 2046.
short	TNF_UNCHANGED (/reference/android/nfc/NdefRecord#TNF_UNCHANGED) Indicates the payload is an intermediate or final chunk of a chunked NDEF Record.
short	TNF_UNKNOWN (/reference/android/nfc/NdefRecord#TNF_UNKNOWN) Indicates the payload type is unknown.

short**TNF_WELL_KNOWN**[\(/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN\)](/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN)

Indicates the type field contains a well-known RTD type name.

Inherited constants

From interface [android.os.Parcelable](/reference/android/os/Parcelable) [\(/reference/android/os/Parcelable\)](/reference/android/os/Parcelable)

int**CONTENTS_FILE_DESCRIPTOR**[\(/reference/android/os/Parcelable#CONTENTS_FILE_DESCRIPTOR\)](/reference/android/os/Parcelable#CONTENTS_FILE_DESCRIPTOR)

Descriptor bit used with [describeContents\(\)](#)

[\(/reference/android/os/Parcelable#describeContents\(\)\)](/reference/android/os/Parcelable#describeContents()): indicates that the Parcelable object's flattened representation includes a file descriptor.

int**PARCELABLE_WRITE_RETURN_VALUE**[\(/reference/android/os/Parcelable#PARCELABLE_WRITE_RETURN_VALUE\)](/reference/android/os/Parcelable#PARCELABLE_WRITE_RETURN_VALUE)

Flag for use with [writeToParcel\(Parcel, int\)](#)

[\(/reference/android/os/Parcelable#writeToParcel\(android.os.Parcel, %20int\)\)](/reference/android/os/Parcelable#writeToParcel(android.os.Parcel,%20int))

: the object being written is a return value, that is the result of a function such as "[Parcelable someFunction\(\)](#)", "[void someFunction\(out Parcelable\)](#)", or "[void someFunction\(inout Parcelable\)](#)".

Fields

public static final**Creator**[\(/reference/android/os/Parcelable.Creator\)](/reference/android/os/Parcelable.Creator)**<NdefRecord**[\(/reference/android/nfc/NdefRecord\)](/reference/android/nfc/NdefRecord)**>****CREATOR** [\(/reference/android/nfc/NdefRecord#CREATOR\)](/reference/android/nfc/NdefRecord#CREATOR)

```
public static final  
byte[]
```

RTD_ALTERNATIVE_CARRIER

([/reference/android/nfc/NdefRecord#RTD_ALTERNATIVE_CARRIER](#))

RTD Alternative Carrier type.

```
public static final  
byte[]
```

RTD_HANDOVER_CARRIER

([/reference/android/nfc/NdefRecord#RTD_HANDOVER_CARRIER](#))

RTD Handover Carrier type.

```
public static final  
byte[]
```

RTD_HANDOVER_REQUEST

([/reference/android/nfc/NdefRecord#RTD_HANDOVER_REQUEST](#))

RTD Handover Request type.

```
public static final  
byte[]
```

RTD_HANDOVER_SELECT

([/reference/android/nfc/NdefRecord#RTD_HANDOVER_SELECT](#))

RTD Handover Select type.

```
public static final  
byte[]
```

RTD_SMART_POSTER

([/reference/android/nfc/NdefRecord#RTD_SMART_POSTER](#))

RTD Smart Poster type.

```
public static final  
byte[]
```

RTD_TEXT ([/reference/android/nfc/NdefRecord#RTD_TEXT](#))

RTD Text type.

```
public static final  
byte[]
```

RTD_URI ([/reference/android/nfc/NdefRecord#RTD_URI](#))

RTD URI type.

Public constructors

NdefRecord ([/reference/android/nfc/NdefRecord#NdefRecord\(byte\[\]\)](#))(**byte[] data**)

*This constructor is deprecated. use **NdefMessage.NdefMessage(byte[])**.
([/reference/android/nfc/NdefMessage#NdefMessage\(byte\[\]\)](#)) instead.*

NdefRecord (/reference/android/nfc/NdefRecord#NdefRecord(short,%20byte[],%20byte[],%20byte[]))
(short tnf, byte[] type, byte[] id, byte[] payload)

Construct an NDEF Record from its component fields.

Public methods

static NdefRecord
(/reference/android/nfc/
NdefRecord)

createApplicationRecord
(/reference/android/nfc/
NdefRecord#createApplicationRecord(java.lang.String))
(String (/reference/java/lang/String) packageName)

Create a new Android Application Record (AAR).

static NdefRecord
(/reference/android/nfc/
NdefRecord)

createExternal
(/reference/android/nfc/NdefRecord#createExternal(java.lang.String,
%20java.lang.String,%20byte[]))
(String (/reference/java/lang/String) domain, String
(/reference/java/lang/String) type, byte[] data)

Create a new NDEF Record containing external (application-specific) data.

static NdefRecord
(/reference/android/nfc/
NdefRecord)

createMime
(/reference/android/nfc/NdefRecord#createMime(java.lang.String,
%20byte[]))
(String (/reference/java/lang/String) mimeType, byte[] mime
Data)

Create a new NDEF Record containing MIME data.

static NdefRecord
(/reference/android/nfc/
NdefRecord)

createTextRecord
(/reference/android/nfc/NdefRecord#createTextRecord(java.lang.String,
%20java.lang.String))
(String (/reference/java/lang/String) languageCode, String
(/reference/java/lang/String) text)

Create a new NDEF record containing UTF-8 text data.

static NdefRecord`(/reference/android/nfc/
NdefRecord)`**createUri**`(/reference/android/nfc/NdefRecord#createUri(android.net.Uri))(Uri
(/reference/android/net/Uri) uri)`

Create a new NDEF Record containing a URI.

static NdefRecord`(/reference/android/nfc/
NdefRecord)`**createUri**`(/reference/android/nfc/NdefRecord#createUri(java.lang.String))
(String (/reference/java/lang/String) uriString)`

Create a new NDEF Record containing a URI.

int**describeContents**`(/reference/android/nfc/NdefRecord#describeContents())()`

Describe the kinds of special objects contained in this Parcelable instance's marshaled representation.

boolean**equals** `(/reference/android/nfc/NdefRecord#equals(java.lang.Object))
(Object (/reference/java/lang/Object) obj)`

Returns true if the specified NDEF Record contains identical tnf, type, id and payload fields.

byte[]**getId** `(/reference/android/nfc/NdefRecord#getId())()`

Returns the variable length ID.

byte[]**getPayload** `(/reference/android/nfc/NdefRecord#getPayload())()`

Returns the variable length payload.

short**getTnf** `(/reference/android/nfc/NdefRecord#getTnf())()`

Returns the 3-bit TNF.

byte[]**getType** `(/reference/android/nfc/NdefRecord#getType())()`

Returns the variable length Type field.

int**hashCode** (/reference/android/nfc/NdefRecord#hashCode())()

Returns a hash code value for the object.

byte[]**toArray** (/reference/android/nfc/NdefRecord#toArray())()*This method was deprecated in API level 16. use***NdefMessage.toArray()***(/reference/android/nfc/NdefMessage#toArray()) instead***String***(/reference/java/lang/String)***toMimeType** (/reference/android/nfc/NdefRecord#toMimeType())()

Map this record to a MIME type, or return null if it cannot be mapped.

String*(/reference/java/lang/String)***toString** (/reference/android/nfc/NdefRecord#toString())()

Returns a string representation of the object.

Uri (/reference/android/net/Uri)**toUri** (/reference/android/nfc/NdefRecord#toUri())()

Map this record to a URI, or return null if it cannot be mapped.

void**writeToParcel***(/reference/android/nfc/NdefRecord#writeToParcel(android.os.Parcel, %20int))***(Parcel** (/reference/android/os/Parcel) **dest**, **int flags**)

Flatten this object in to a Parcel.

Inherited methods

From class **java.lang.Object** (/reference/java/lang/Object)**Object** (/reference/java/lang/Object)**clone** (/reference/java/lang/Object#clone())()

Creates and returns a copy of this object.

boolean**equals***(/reference/java/lang/Object#equals(java.lang.Object))***(Object** (/reference/java/lang/Object) **obj**)

Indicates whether some other object is "equal to" this one.

void

finalize (/reference/java/lang/Object#finalize()) ()

Called by the garbage collector on an object when garbage collection determines that there are no more references to the object.

final Class (/reference/java/lang/Class)<?>

getClass (/reference/java/lang/Object#getClass()) ()

Returns the runtime class of this **Object**.

int

hashCode (/reference/java/lang/Object#hashCode()) ()

Returns a hash code value for the object.

final void

notify (/reference/java/lang/Object#notify()) ()

Wakes up a single thread that is waiting on this object's monitor.

final void

notifyAll (/reference/java/lang/Object#notifyAll()) ()

Wakes up all threads that are waiting on this object's monitor.

String (/reference/java/lang/String)

toString (/reference/java/lang/Object#toString()) ()

Returns a string representation of the object.

final void

wait (/reference/java/lang/Object#wait(long,%20int))
(**long timeoutMillis**, **int nanos**)

Causes the current thread to wait until it is awakened, typically by being *notified* or *interrupted*, or until a certain amount of real time has elapsed.

final void

wait (/reference/java/lang/Object#wait(long)) (**long timeoutMillis**)

Causes the current thread to wait until it is awakened, typically by being *notified* or *interrupted*, or until a certain amount of real time has elapsed.

final void

wait (/reference/java/lang/Object#wait()) ()

Causes the current thread to wait until it is awakened, typically by being *notified* or *interrupted*.

From interface **android.os.Parcelable** (/reference/android/os/Parcelable)

abstract int

describeContents

(/reference/android/os/Parcelable#describeContents()) ()

Describe the kinds of special objects contained in this Parcelable instance's marshaled representation.

abstract void

writeToParcel

(/reference/android/os/Parcelable#writeToParcel(android.os.Parcel, %20int))

(**Parcel** (/reference/android/os/Parcel) **dest**, **int flags**)

Flatten this object in to a Parcel.

Constants

TNF_ABSOLUTE_URI Added in API level 9 (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static final short TNF_ABSOLUTE_URI
```

Indicates the type field contains an absolute-URI BNF construct defined by RFC 3986.

When creating new records prefer **createUri(Uri)**

(/reference/android/nfc/NdefRecord#createUri(android.net.Uri)), since it offers more compact URI encoding (#RTD_URI allows compression of common URI prefixes).

See also:

[`createUri\(Uri\)`](#) ([/reference/android/nfc/NdefRecord#createUri\(android.net.Uri\)](/reference/android/nfc/NdefRecord#createUri(android.net.Uri)))

Constant Value: 3 (0x00000003)

TNF_EMPTY

Added in [API level 9](#) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public static final short TNF_EMPTY
```

Indicates the record is empty.

Type, id and payload fields are empty in a TNF_EMPTY record.

Constant Value: 0 (0x00000000)

TNF_EXTERNAL_TYPE

Added in [API level 9](#) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public static final short TNF_EXTERNAL_TYPE
```

Indicates the type field contains an external type name.

Used to encode custom payloads. When creating new records use the helper

[`createExternal\(String, String, byte\)`](#).

([/reference/android/nfc/NdefRecord#createExternal\(java.lang.String,%20java.lang.String,%20byte\[\]\)](/reference/android/nfc/NdefRecord#createExternal(java.lang.String,%20java.lang.String,%20byte[]))).

The external-type RTD format is specified in NFCForum-TS-RTD_1.0.

Note this TNF should not be used with RTD_TEXT or RTD_URI constants. Those are well known RTD constants, not external RTD constants.

See also:

createExternal(String, String, byte)

(/reference/android/nfc/NdefRecord#createExternal(java.lang.String,%20java.lang.String,%20byte[]))

Constant Value: 4 (0x00000004)

TNF_MIME_MEDIA

Added in [API level 9](#) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static final short TNF_MIME_MEDIA
```

Indicates the type field contains a media-type BNF construct, defined by RFC 2046.

Use this with MIME type names such as "image/jpeg", or using the helper

createMime(String, byte)

(/reference/android/nfc/NdefRecord#createMime(java.lang.String,%20byte[])).

See also:**createMime(String, byte)**

(/reference/android/nfc/NdefRecord#createMime(java.lang.String,%20byte[]))

Constant Value: 2 (0x00000002)

TNF_UNCHANGED

Added in [API level 9](#) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static final short TNF_UNCHANGED
```

Indicates the payload is an intermediate or final chunk of a chunked NDEF Record.

TNF_UNCHANGED can not be used with this class since all [NdefRecord](#)

(/reference/android/nfc/NdefRecord)s are already unchunked, however they may appear in the binary format.

Constant Value: 6 (0x00000006)

TNF_UNKNOWN

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static final short TNF_UNKNOWN
```

Indicates the payload type is unknown.

NFC Forum explains this should be treated similarly to the "application/octet-stream" MIME type. The payload type is not explicitly encoded within the record.

The type field is empty in an TNF_UNKNOWN record.

Constant Value: 5 (0x00000005)

TNF_WELL_KNOWN

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static final short TNF_WELL_KNOWN
```

Indicates the type field contains a well-known RTD type name.

Use this tnf with RTD types such as [RTD_TEXT](/reference/android/nfc/NdefRecord#RTD_TEXT) (/reference/android/nfc/NdefRecord#RTD_TEXT), [RTD_URI](/reference/android/nfc/NdefRecord#RTD_URI) (/reference/android/nfc/NdefRecord#RTD_URI).

The RTD type name format is specified in NFCForum-TS-RTD_1.0.

See also:

[RTD_URI](/reference/android/nfc/NdefRecord#RTD_URI) (/reference/android/nfc/NdefRecord#RTD_URI)

[RTD_TEXT](/reference/android/nfc/NdefRecord#RTD_TEXT) (/reference/android/nfc/NdefRecord#RTD_TEXT)

RTD_SMART_POSTER (/reference/android/nfc/NdefRecord#RTD_SMART_POSTER)

createUri(Uri) ([/reference/android/nfc/NdefRecord#createUri\(android.net.Uri\)](/reference/android/nfc/NdefRecord#createUri(android.net.Uri)))

Constant Value: 1 (0x00000001)

Fields

CREATOR

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public static final Creator <NdefRecord> (/reference/android/os/Parcelable.Creator)
```

RTD_ALTERNATIVE_CARRIER

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public static final byte[] RTD_ALTERNATIVE_CARRIER
```

RTD Alternative Carrier type. For use with TNF_WELL_KNOWN.

See also:

TNF_WELL_KNOWN (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN)

RTD_HANDOVER_CARRIER

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public static final byte[] RTD_HANDOVER_CARRIER
```

RTD Handover Carrier type. For use with `TNF_WELL_KNOWN`.

See also:

[`TNF\_WELL\_KNOWN`](/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN) (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN)

RTD_HANOVER_REQUEST Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static final byte[] RTD_HANOVER_REQUEST
```

RTD Handover Request type. For use with `TNF_WELL_KNOWN`.

See also:

[`TNF\_WELL\_KNOWN`](/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN) (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN)

RTD_HANOVER_SELECT Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static final byte[] RTD_HANOVER_SELECT
```

RTD Handover Select type. For use with `TNF_WELL_KNOWN`.

See also:

[`TNF\_WELL\_KNOWN`](/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN) (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN)

RTD_SMART_POSTER Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static final byte[] RTD_SMART_POSTER
```

RTD Smart Poster type. For use with `TNF_WELL_KNOWN`.

See also:

[`TNF\_WELL\_KNOWN`](#) (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN)

RTD_TEXT

Added in [API level 9](#) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static final byte[] RTD_TEXT
```

RTD Text type. For use with `TNF_WELL_KNOWN`.

See also:

[`TNF\_WELL\_KNOWN`](#) (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN)

RTD_URI

Added in [API level 9](#) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static final byte[] RTD_URI
```

RTD URI type. For use with `TNF_WELL_KNOWN`.

See also:

[`TNF\_WELL\_KNOWN`](#) (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN)

Public constructors

NdefRecord

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public NdefRecord (byte[] data)
```

This constructor is deprecated.

use [NdefMessage.NdefMessage\(byte\[\]\)](/reference/android/nfc/NdefMessage#NdefMessage(byte[])) ([/reference/android/nfc/NdefMessage#NdefMessage\(byte\[\]\)](/reference/android/nfc/NdefMessage#NdefMessage(byte[]))) instead.

Construct an NDEF Record from raw bytes.

This method is deprecated, use [NdefMessage.NdefMessage\(byte\[\]\)](/reference/android/nfc/NdefMessage#NdefMessage(byte[])) ([/reference/android/nfc/NdefMessage#NdefMessage\(byte\[\]\)](/reference/android/nfc/NdefMessage#NdefMessage(byte[]))) instead. This is because it does not make sense to parse a record: the NDEF binary format is only defined for a message, and the record flags MB and ME do not make sense outside of the context of an entire message.

This implementation will attempt to parse a single record by ignoring the MB and ME flags, and otherwise following the rules of [NdefMessage.NdefMessage\(byte\[\]\)](/reference/android/nfc/NdefMessage#NdefMessage(byte[])) ([/reference/android/nfc/NdefMessage#NdefMessage\(byte\[\]\)](/reference/android/nfc/NdefMessage#NdefMessage(byte[]))).

Parameters

data	byte : raw bytes to parse
-------------	----------------------------------

Throws

<u>FormatException</u>	if the data cannot be parsed into a valid record
--	--

(/reference/android/nfc/
FormatException)

NdefRecord

Added in [API level 9](#) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public NdefRecord (short tnf,  
                  byte[] type,  
                  byte[] id,  
                  byte[] payload)
```

Construct an NDEF Record from its component fields.

Recommend to use helpers such as {[createUrl](#)} or {[createExternal\(String, String, byte\)](#)}

(/reference/android/nfc/NdefRecord#createExternal(java.lang.String,%20java.lang.String,%20byte[]))

where possible, since they perform stricter validation that the record is correctly formatted as per NDEF specifications. However if you know what you are doing then this constructor offers the most flexibility.

An [NdefRecord](#) (/reference/android/nfc/NdefRecord) represents a logical (complete) record, and cannot represent NDEF Record chunks.

Basic validation of the tnf, type, id and payload is performed as per the following rules:

- The tnf paramter must be a 3-bit value.
- Records with a tnf of [TNF_EMPTY](#) (/reference/android/nfc/NdefRecord#TNF_EMPTY) cannot have a type, id or payload.
- Records with a tnf of [TNF_UNKNOWN](#) (/reference/android/nfc/NdefRecord#TNF_UNKNOWN) or 0x07 cannot have a type.
- Records with a tnf of [TNF_UNCHANGED](#) (/reference/android/nfc/NdefRecord#TNF_UNCHANGED) are not allowed since this class only represents complete (unchunked) records.

This minimal validation is specified by NFCForum-TS-NDEF_1.0 section 3.2.6 (Type Name Format).

If any of the above validation steps fail then `IllegalArgumentException` (</reference/java/lang/IllegalArgumentException>) is thrown.

Deep inspection of the type, id and payload fields is not performed, so it is possible to create NDEF Records that conform to section 3.2.6 but fail other more strict NDEF specification requirements. For example, the payload may be invalid given the tnf and type.

To omit a type, id or payload field, set the parameter to an empty byte array or null.

Parameters

<code>tnf</code>	<code>short</code> : a 3-bit TNF constant
<code>type</code>	<code>byte</code> : byte array, containing zero to 255 bytes, or null
<code>id</code>	<code>byte</code> : byte array, containing zero to 255 bytes, or null
<code>payload</code>	<code>byte</code> : byte array, containing zero to $(2^{32} - 1)$ bytes, or null

Throws

`IllegalArgumentException` (</reference/java/lang/IllegalArgumentException>) if a valid record cannot be created

Public methods

createApplicationRecord added in API level 14 (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public static NdefRecord (/reference/android/nfc/NdefRecord) createApplicationRecord (St
```

Create a new Android Application Record (AAR).

This record indicates to other Android devices the package that should be used to handle the entire NDEF message. You can embed this record anywhere into your message to ensure that the intended package receives the message.

When an Android device dispatches an [NdefMessage](#) (/reference/android/nfc/NdefMessage) containing one or more Android application records, the applications contained in those records will be the preferred target for the [NfcAdapter.ACTION_NDEF_DISCOVERED](#) (/reference/android/nfc/NfcAdapter#ACTION_NDEF_DISCOVERED) intent, in the order in which they appear in the message. This dispatch behavior was first added to Android in Ice Cream Sandwich.

If none of the applications have a are installed on the device, a Market link will be opened to the first application.

Note that Android application records do not overrule applications that have called [NfcAdapter.enableForegroundDispatch](#)

```
(/reference/android/nfc/NfcAdapter#enableForegroundDispatch(android.app.Activity,  
%20android.app.PendingIntent,%20android.content.IntentFilter[],%20java.lang.String[][]))
```

.

Parameters

packageName	String : Android package name
-----------------------------	---

Returns

NdefRecord (/reference/android/nfc/ NdefRecord)	Android application NDEF record
---	---------------------------------

createExternal

Added in [API level 16](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public static NdefRecord (/reference/android/nfc/NdefRecord) createExternal (String (/reference/java/lang/String) type,  
byte[] data)
```

Create a new NDEF Record containing external (application-specific) data.

Use this method to encode application specific data into an NDEF Record. The data is typed by a domain name (usually your Android package name) and a domain-specific type. This data is packaged into a "NFC Forum External Type" NDEF Record.

NFC Forum requires that the domain and type used in an external record are treated as case insensitive, however Android intent filtering is always case sensitive. So this method will force the domain and type to lower-case before creating the NDEF Record.

The unchecked exception [IllegalArgumentException](/reference/java/lang/IllegalArgumentException) (</reference/java/lang/IllegalArgumentException>) will be thrown if the domain and type have serious problems, for example if either field is empty, so always catch this exception if you are passing user-generated data into this method.

There are no such restrictions on the payload data.

For efficiency, This method might not make an internal copy of the data byte array, so take care not to modify the data byte array while still using the returned NdefRecord. Reference specification: NFCForum-TS-RTD_1.0

Parameters

domain	String : domain-name of issuing organization
---------------	---

type	String : domain-specific type of data
-------------	--

data**byte:** payload as bytes

Returns

NdefRecord[\(/reference/android/nfc/NdefRecord\)](/reference/android/nfc/NdefRecord)

Throws

IllegalArgumentException if either domain or type are empty or invalid[\(/reference/java/lang/IllegalArgumentException\)](/reference/java/lang/IllegalArgumentException)

createMime

Added in [API level 16](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public static NdefRecord (/reference/android/nfc/NdefRecord) createMime (String (/reference.  
byte[] mimeType)
```

Create a new NDEF Record containing MIME data.

Use this method to encode MIME-typed data into an NDEF Record, such as "text/plain", or "image/jpeg".

The mimeType parameter will be normalized with [Intent.normalizeMimeType](#) ([/reference/android/content/Intent#normalizeMimeType\(java.lang.String\)](/reference/android/content/Intent#normalizeMimeType(java.lang.String))) to follow Android best practices for intent filtering, for example to force lower-case. However the unchecked exception [IllegalArgumentException](#) (</reference/java/lang/IllegalArgumentException>) may be thrown if the mimeType parameter has serious problems, for example if it is empty, so always

catch this exception if you are passing user-generated data into this method.

For efficiency, This method might not make an internal copy of the mimeType byte array, so take care not to modify the mimeType byte array while still using the returned NdefRecord.

Parameters

mimeType	String : a valid MIME type
-----------------	-----------------------------------

mimeTypeData	byte : MIME data as bytes
---------------------	----------------------------------

Returns

NdefRecord (/reference/android/nfc/NdefRecord)	an NDEF Record containing the MIME-typed data
--	---

Throws

IllegalArgumentException (/reference/java/lang/IllegalArgumentException)	if the mimeType is empty or invalid
--	-------------------------------------

createTextRecord Added in [API level 21](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static NdefRecord (/reference/android/nfc/NdefRecord) createTextRecord (String (/re
String (/reference/java/lang/String) text)
```

Create a new NDEF record containing UTF-8 text data.

The caller can either specify the language code for the provided text, or otherwise the language code corresponding to the current default locale will be used. Reference specification: NFCForum-TS-RTD_Text_1.0

Parameters

languageCode	String : The languageCode for the record. If locale is empty or null, the language code of the current default locale will be used.
text	String : The text to be encoded in the record. Will be represented in UTF-8 format.

Returns

NdefRecord
(</reference/android/nfc/NdefRecord>)

Throws

IllegalArgumentException if text is null
(</reference/java/lang/IllegalArgumentException>)

createUrl

Added in [API level 14](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)


```
public static NdefRecord (/reference/android/nfc/NdefRecord) createUri (Uri (/reference/andr
```

Create a new NDEF Record containing a URI.

Use this method to encode a URI (or URL) into an NDEF Record.

Uses the well known URI type representation: [TNF_WELL_KNOWN](#) (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN) and [RTD_URI](#) (/reference/android/nfc/NdefRecord#RTD_URI). This is the most efficient encoding of a URI into NDEF.

The uri parameter will be normalized with [Uri.normalizeScheme](#) (/reference/android/net/Uri#normalizeScheme()) to set the scheme to lower case to follow Android best practices for intent filtering. However the unchecked exception [IllegalArgumentException](#) (/reference/java/lang/IllegalArgumentException) may be thrown if the uri parameter has serious problems, for example if it is empty, so always catch this exception if you are passing user-generated data into this method.

Reference specification: NFCForum-TS-RTD_URI_1.0

Parameters

uri	Uri : URI to encode.
---------------------	--------------------------------------

Returns

NdefRecord (/reference/android/nfc/NdefRecord)	an NDEF Record containing the URI
---	-----------------------------------

Throws

`IllegalArgumentException` if the uri is empty or invalid
(/reference/java/lang/
`IllegalArgumentException`)

createUri

Added in [API level 14](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public static NdefRecord createUri (String uriString)
```

Create a new NDEF Record containing a URI.

Use this method to encode a URI (or URL) into an NDEF Record.

Uses the well known URI type representation: [TNF_WELL_KNOWN](/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN) (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN) and [RTD_URI](/reference/android/nfc/NdefRecord#RTD_URI) (/reference/android/nfc/NdefRecord#RTD_URI). This is the most efficient encoding of a URI into NDEF.

The uriString parameter will be normalized with [Uri.normalizeScheme](/reference/android/net/Uri#normalizeScheme) (/reference/android/net/Uri#normalizeScheme()) to set the scheme to lower case to follow Android best practices for intent filtering. However the unchecked exception [IllegalArgumentException](/reference/java/lang/IllegalArgumentException) (/reference/java/lang/IllegalArgumentException) may be thrown if the uriString parameter has serious problems, for example if it is empty, so always catch this exception if you are passing user-generated data into this method.

Reference specification: NFCForum-TS-RTD_URI_1.0

Parameters

<code>uriString</code>	<code>String</code> : string URI to encode.
-------------------------------	--

Returns

NdefRecord an NDEF Record containing the URI
(</reference/android/nfc/NdefRecord>)

Throws

IllegalArgumentException if the uriString is empty or invalid
(</reference/java/lang/IllegalArgumentException>)

describeContents Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public int describeContents ()
```

Describe the kinds of special objects contained in this Parcelable instance's marshaled representation. For example, if the object will include a file descriptor in the output of [`writeToParcel\(Parcel,int\)`](/reference/android/os/Parcelable#writeToParcel(android.os.Parcel,%20int)) ([`writeToParcel\(android.os.Parcel,%20int\)`](/reference/android/os/Parcelable#writeToParcel(android.os.Parcel,%20int))), the return value of this method must include the [`CONTENTS\_FILE\_DESCRIPTOR`](/reference/android/os/Parcelable#CONTENTS_FILE_DESCRIPTOR) ([`CONTENTS\_FILE\_DESCRIPTOR`](/reference/android/os/Parcelable#CONTENTS_FILE_DESCRIPTOR)) bit.

Returns

`int` a bitmask indicating the set of special object types marshaled by this Parcelable object instance.
Value is either `0` or

- [`CONTENTS\_FILE\_DESCRIPTOR`](/reference/android/os/Parcelable#CONTENTS_FILE_DESCRIPTOR)

/reference/android/os/Parcelable#CONTENTS_FILE_DESCRIPTOR

equals

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public boolean equals (Object (/reference/java/lang/Object) obj)
```

Returns true if the specified NDEF Record contains identical tnf, type, id and payload fields.

Parameters

<code>obj</code>	<code>Object</code> : This value may be <code>null</code> .
------------------	---

Returns

<code>boolean</code>	<code>true</code> if this object is the same as the obj argument; <code>false</code> otherwise.
----------------------	---

getId

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public byte[] getId ()
```

Returns the variable length ID.

Returns an empty byte array if this record does not have an id field.

Returns

`byte[]`

getPayload

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public byte[] getPayload ()
```

Returns the variable length payload.

Returns an empty byte array if this record does not have a payload field.

Returns

`byte[]`

getTnf

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public short getTnf ()
```

Returns the 3-bit TNF.

TNF is the top-level type.

Returns

short

getType

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public byte[] getType ()
```

Returns the variable length Type field.

This should be used in conjunction with the TNF field to determine the payload format.

Returns an empty byte array if this record does not have a type field.

Returns

byte[]

hashCode

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public int hashCode ()
```

Returns a hash code value for the object. This method is supported for the benefit of hash tables such as those provided by [HashMap](/reference/java/util/HashMap) (/reference/java/util/HashMap).

The general contract of **hashCode** is:

- Whenever it is invoked on the same object more than once during an execution of a Java application, the **hashCode** method must consistently return the same integer,

provided no information used in `equals` comparisons on the object is modified. This integer need not remain consistent from one execution of an application to another execution of the same application.

- If two objects are equal according to the `equals` ([`\(/reference/java/lang/Object#equals\(java.lang.Object\)\)`](/reference/java/lang/Object#equals(java.lang.Object))) method, then calling the `hashCode` method on each of the two objects must produce the same integer result.
- It is *not* required that if two objects are unequal according to the `equals` ([`\(/reference/java/lang/Object#equals\(java.lang.Object\)\)`](/reference/java/lang/Object#equals(java.lang.Object))) method, then calling the `hashCode` method on each of the two objects must produce distinct integer results. However, the programmer should be aware that producing distinct integer results for unequal objects may improve the performance of hash tables.

Returns

`int` a hash code value for this object.

toByteArray

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) ([`\(/guide/topics/manifest/uses-sdk-element#ApiLevels\)`](/guide/topics/manifest/uses-sdk-element#ApiLevels))

Deprecated in [API level 16](/guide/topics/manifest/uses-sdk-element#ApiLevels) ([`\(/guide/topics/manifest/uses-sdk-element#ApiLevels\)`](/guide/topics/manifest/uses-sdk-element#ApiLevels))

```
public byte[] toByteArray ()
```

This method was deprecated in API level 16.

use `NdefMessage.toByteArray()` ([`\(/reference/android/nfc/NdefMessage#toByteArray\(\)\)`](/reference/android/nfc/NdefMessage#toByteArray())) instead

Return this NDEF Record as a byte array.

This method is deprecated, use `NdefMessage.toByteArray` ([`\(/reference/android/nfc/NdefMessage#toByteArray\(\)\)`](/reference/android/nfc/NdefMessage#toByteArray())) instead. This is because the NDEF binary format is not defined for a record outside of the context of a message: the MB and ME flags cannot be set without knowing the location inside a message.

This implementation will attempt to serialize a single record by always setting the MB and ME flags (in other words, assume this is a single-record NDEF Message).

Returns

`byte[]`

toMimeType

Added in [API level 16](#) ([/guide/topics/manifest/uses-sdk-element#ApiLevels](#))

```
public String (/reference/java/lang/String) toMimeType ()
```

Map this record to a MIME type, or return null if it cannot be mapped.

Currently this method considers all [TNF_MIME_MEDIA](#) ([/reference/android/nfc/NdefRecord#TNF_MIME_MEDIA](#)) records to be MIME records, as well as some [TNF_WELL_KNOWN](#) ([/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN](#)) records such as [RTD_TEXT](#) ([/reference/android/nfc/NdefRecord#RTD_TEXT](#)). If this is a MIME record then the MIME type as string is returned, otherwise null is returned.

This method does not perform validation that the MIME type is actually valid. It always attempts to return a string containing the type if this is a MIME record.

The returned MIME type will be normalized to lower-case using [Intent.normalizeMimeType](#) ([/reference/android/content/Intent#normalizeMimeType\(java.lang.String\)](#)).

The MIME payload can be obtained using [getPayload\(\)](#) ([/reference/android/nfc/NdefRecord#getPayload\(\)](#)).

Returns

String (</reference/java/lang/String>) MIME type as a string, or null if this is not a MIME record

toString Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public String (/reference/java/lang/String) toString ()
```

Returns a string representation of the object.

Returns

String (</reference/java/lang/String>) a string representation of the object.

toUri Added in [API level 16](/guide/topics/manifest/uses-sdk-element#ApiLevels) (</guide/topics/manifest/uses-sdk-element#ApiLevels>)

```
public Uri (/reference/android/net/Uri) toUri ()
```

Map this record to a URI, or return null if it cannot be mapped.

Currently this method considers the following to be URI records:

- [TNF_ABSOLUTE_URI](/reference/android/nfc/NdefRecord#TNF_ABSOLUTE_URI) (/reference/android/nfc/NdefRecord#TNF_ABSOLUTE_URI) records.
- [TNF_WELL_KNOWN](/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN) (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN) with a type of [RTD_URI](/reference/android/nfc/NdefRecord#RTD_URI) (/reference/android/nfc/NdefRecord#RTD_URI).
- [TNF_WELL_KNOWN](/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN) (/reference/android/nfc/NdefRecord#TNF_WELL_KNOWN) with a type of

RTD_SMART_POSTER (/reference/android/nfc/NdefRecord#RTD_SMART_POSTER) and containing a URI record in the NDEF message nested in the payload.

- **TNF_EXTERNAL_TYPE** (/reference/android/nfc/NdefRecord#TNF_EXTERNAL_TYPE) records.

If this is not a URI record by the above rules, then null is returned.

This method does not perform validation that the URI is actually valid: it always attempts to create and return a URI if this record appears to be a URI record by the above rules.

The returned URI will be normalized to have a lower case scheme using

Uri.normalizeScheme (/reference/android/net/Uri#normalizeScheme()).

Returns

Uri (/reference/android/net/Uri) URI, or null if this is not a URI record

writeToParcel

Added in [API level 9](/guide/topics/manifest/uses-sdk-element#ApiLevels) (/guide/topics/manifest/uses-sdk-element#ApiLevels)

```
public void writeToParcel (Parcel (/reference/android/os/Parcel) dest,  
                           int flags)
```

Flatten this object in to a Parcel.

Parameters

dest

Parcel: The Parcel in which the object should be written.
This value cannot be **null**.

flags

int: Additional flags about how the object should be written. May be 0 or **Parcelable.PARCELABLE_WRITE_RETURN_VALUE**

(/reference/android/os/Parcelable#PARCELABLE_WRITE_RETURN_VALUE)

Value is either `0` or a combination of the following:

- **[Parcelable.PARCELABLE_WRITE_RETURN_VALUE](/reference/android/os/Parcelable#PARCELABLE_WRITE_RETURN_VALUE)**
(/reference/android/os/Parcelable#PARCELABLE_WRITE_RETURN_VALUE)

Content and code samples on this page are subject to the licenses described in the [Content License \(/license\)](/license).
Java and OpenJDK are trademarks or registered trademarks of Oracle and/or its affiliates.

Last updated 2026-02-26 UTC.